实验 6. 随机化算法的初步实践-N 后等问题

实验内容

本实验要求基于算法设计与分析的一般过程（即待求解问题的描述、算法设计、算法描述、算法正确性证明、算法分析、算法实现与测试），通过随机化方法（包括拉斯维加斯法）在实际问题求解实践中，加深理解其基本原理和思想以及求解步骤。求解的问题为 N 后问题。**作为挑战：可以考虑其他随机化算法（如随机快排、蒙特卡洛法估计问题的平均情况下的复杂性/搜索树的规模等）。**

实验目的

- 理解随机化算法的核心思想以及求解过程；
- 掌握几种随机化算法（即数值随机化算法、蒙特卡洛算法、拉斯维加斯算法以及舍伍德算法）；
- 从算法分析与设计的角度，对 N 后问题等的随机化算法有更进一步的理解。

环境要求

无特别要求。算法实现可以自由选择 C, C++, Java, Python 等。

实验步骤

步骤 1：理解问题，给出问题的描述。

步骤 2：算法设计，包括策略与数据结构的选择

步骤 3：描述算法。希望采用源代码以外的形式，如伪代码或流程图等；

```
bool QueesnsLV()
while k less than or equal n
    for i from 1 to MAXCOUNT
        srand((unsigned)time(NULL) + i)
        res[k] = rand() % n + 1
        if judge(k)
            break
    if i less or equal n
        k++
    else
        break
return k great than n;
```

步骤 4：算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明（此处可以略去）；

步骤 5：算法复杂性分析，包括时间复杂性和空间复杂性。可以基于蒙特卡洛算法来估计平均情况下的算法复杂性；

步骤 6：算法实现与测试。附上代码或以附件的形式提交，同时贴上算法运行结果截图；
步骤 7：技术上、分析过程中等各种心得体会与备忘，需要言之有物。
说明：步骤 1-6 在“实验结果”一节中描述，步骤 7 在“实验总结”一节中描述。

实验结果

步骤 1：理解问题，给出问题的描述。

在 n×n 格的棋盘上，放置 n 个皇后，任意两个皇后不会相互攻击。

步骤 2：算法设计，包括策略与数据结构的选择

从棋盘的第一行开始，首先搜索该行皇后可以存放的有效位置，然后从这些有效位置中随机挑选一个位置放置皇后。随着放置的皇后数越来越多，失败的概率也越来越大。该算法中使用一个数组存放每行皇后存放的有效位置，用另一个数组存放每行皇后放置位置，使用 while 循环调用拉斯维加斯算法，记录失败次数直到成功得到一个有效解，然后输出相关信息。

步骤 3：描述算法。希望采用源代码以外的形式，如伪代码或流程图等；

步骤 4：算法的正确性证明。需要这个环节，在理解的基础上对算法的正确性给予证明（此处可以略去）；

步骤 5：算法复杂性分析，包括时间复杂性和空间复杂性。可以基于蒙特卡洛算法来估计平均情况下的算法复杂性；

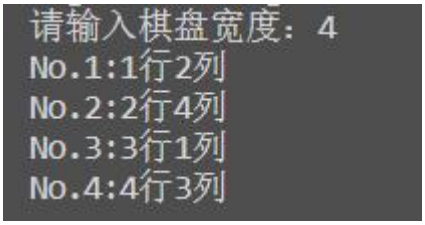
设 p(x)是对该算法的输入 x 调用它获得一个解的概率，s(x)和 e(x)分别是算法对于具体 实例 x 成功和失败所需的平均时间,t(x)为找到具体实例 x 的一个解所需的平均时间。由于 p(x)>0，进行 n 次试验，成功的次数为 np(x)，不成功的次数为 n(1-p(x))，所耗用的总时间为 np(x)s(x)+n(1-p(x))e(x),则 t(x)为：

$$\frac{np(x)s(x)+n\big(1-p(x)\big)e(x)}{np(x)}=s(x)+\frac{1-p(x)}{p(x)}e(x)$$

步骤 6：算法实现与测试。附上代码或以附件的形式提交，同时贴上算法运行结果截图；

```
#include <bits/stdc++.h>
using namespace std;
#define MAXN 1e8;

int res[10];
int n;

bool judge(int k) {
    for (int i = 1; i < k; i++) {
        if (res[i] == res[k] || abs(res[i] - res[k]) == abs(k - i))
            return false;
    }
    return true;
}

bool QueesnsLV() {
    int k = 1, count = MAXN;
```

```
    while (k <= n) {
        int i;
        for (i = 1; i <= count; i++) {
            srand((unsigned)time(NULL) + i);
            res[k] = rand() % n + 1;
            if (judge(k))
                break;
        }
        if (i <= count)
            k++;
        else
            break;
    }

    return k > n;
}

int main() {
    cout << "请输入棋盘宽度：";
    cin >> n;
    if (QueesnsLV()) {
        for (int i = 1; i <= n; i++) {
            cout << "No." << i << ":" << i << "行" << res[i] << "列" << endl;
        }
    } else {
        cout << "在限定次数中未得到问题的解" << endl;
    }

    return 0;
}
```



步骤 7：技术上、分析过程中等各种心得体会与备忘，需要言之有物。

实验总结

在本次实验中，练习了拉斯维加斯算法，了解到了概率算法优点：省时、低复杂度、容易实现